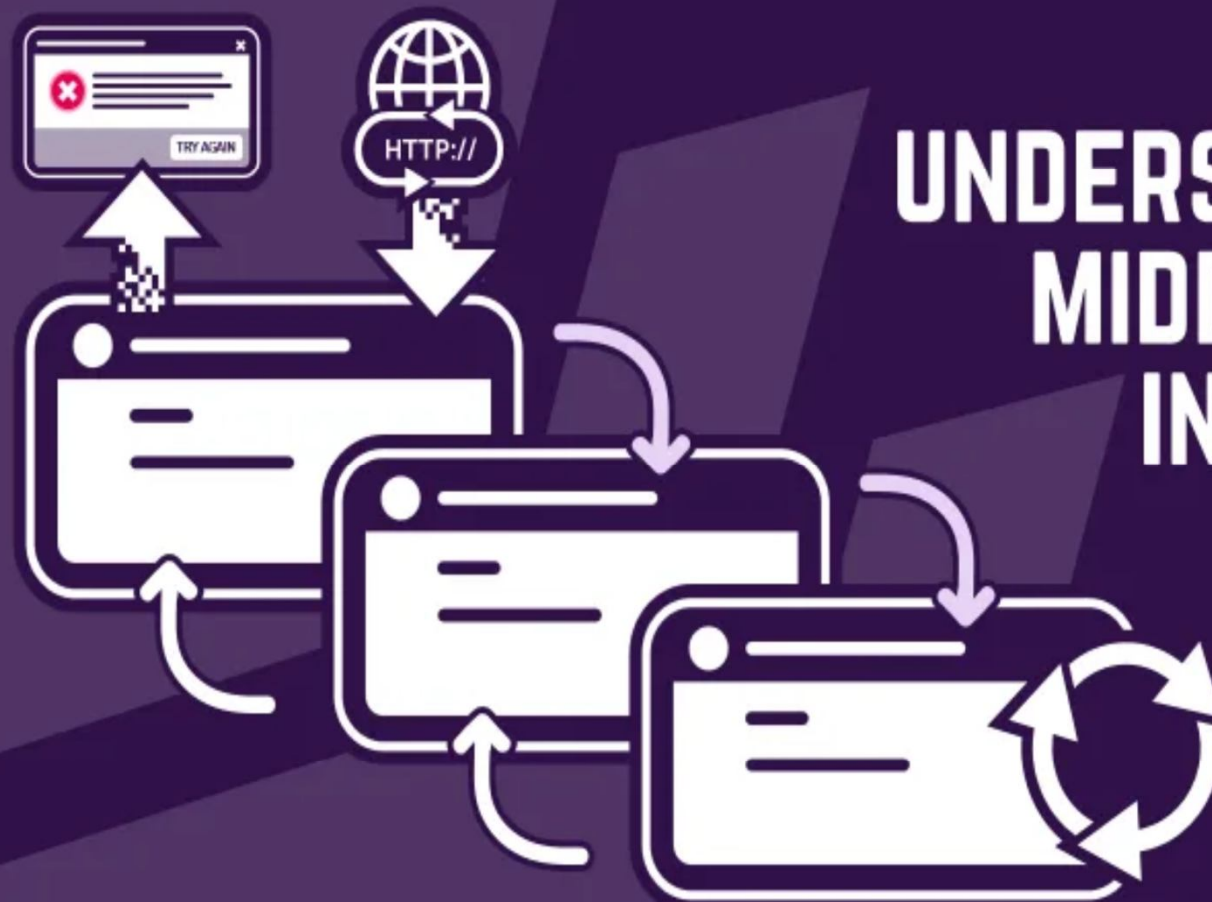


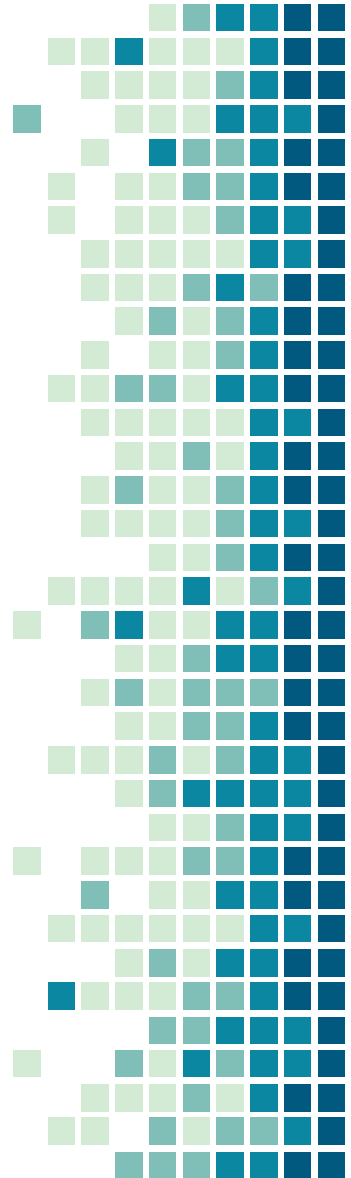


UNDERSTANDING MIDDLEWARE IN ASP.NET CORE

By Achyut V. Kendre

Today's Learning.

- Startup Class Flash Back.
- Configure Service and Configure Method.
- What is Request Pipeline?
- What is Middleware?
- How Middleware Works?
- Run & Use Method?



Startup Class Flash Back

- The ASP.NET Core 3.1, and ASP.NET Core 5 Applications must include a Startup Class.
- It is like the Global.asax file our traditional .NET Application.
- As the name suggests, it is executed first when the application starts.
- The Startup class can be configured using `UseStartup<T>()` Extension method at the time of Configuring the Host in the `Main()` Method of the Program class.



Startup Class Flash Back

- The name “Startup” is by the ASP.NET Core Convention.
- However, you can give any name to the Startup class, just specify it as the generic parameter in the UseStartup<T>() method.
- Startup class must includes two public methods: ConfigureServices and Configure. The Startup class must include the Configure method and can optionally include the ConfigureServices method.

Configure Services Method

- ASP.NET Core built from scratch to Support Dependency Injections.
- It includes the built-in IoC container to provide dependent objects using constructors.
- The ConfigureServices method is a place where you can register your dependent classes with the built-in IoC container.
- After registering the dependent classes, it can be used anywhere in the application.

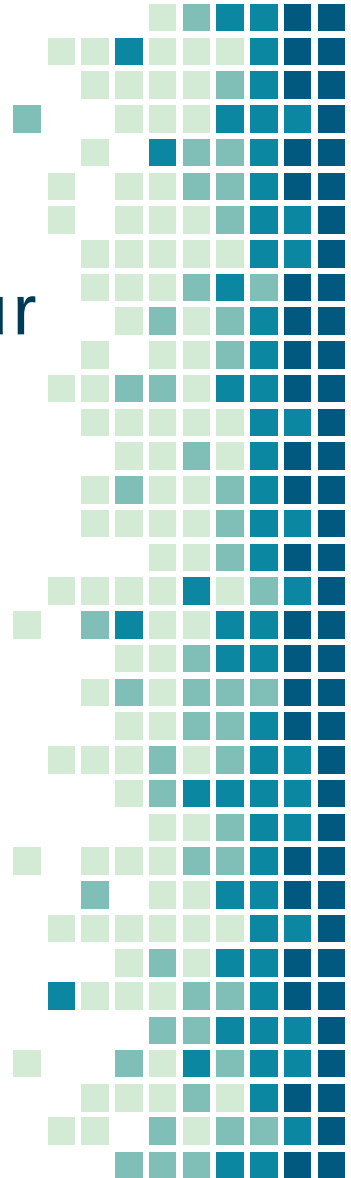


Configure Services Method

- constructor of a class where you want to use it. You just need to include it in the parameter of the
- The IoC container will inject it automatically.
- ASP.NET Core Refers to the dependent class as a Service. So, whenever you read “Service” then understand it as a class that is going to be used in some other class.
- The ConfigureServices method includes the IServiceCollection parameter to register services to the IoC container.

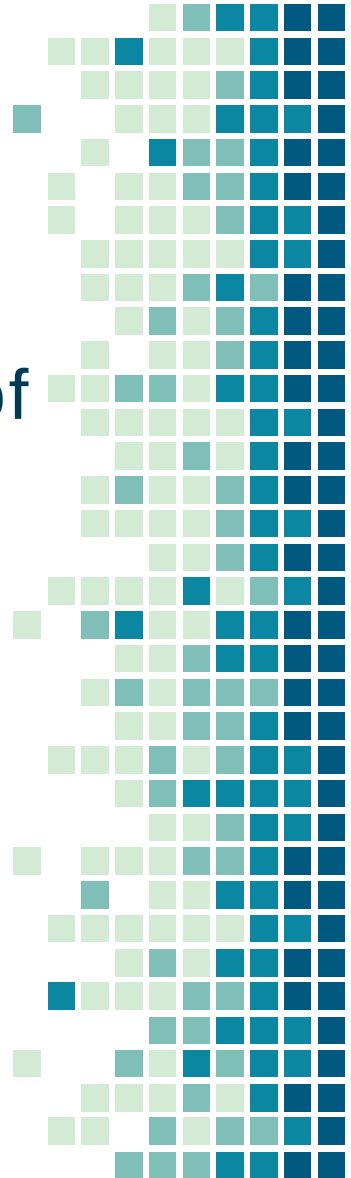
Configure() Method

- The Configure method is a place where we can configure the application request pipeline for our ASP.NET Core application using the `IApplicationBuilder` instance that is provided by the built-in IoC container.
- ASP.NET Core introduced the middleware components to define a request pipeline, which will be executed on every request.

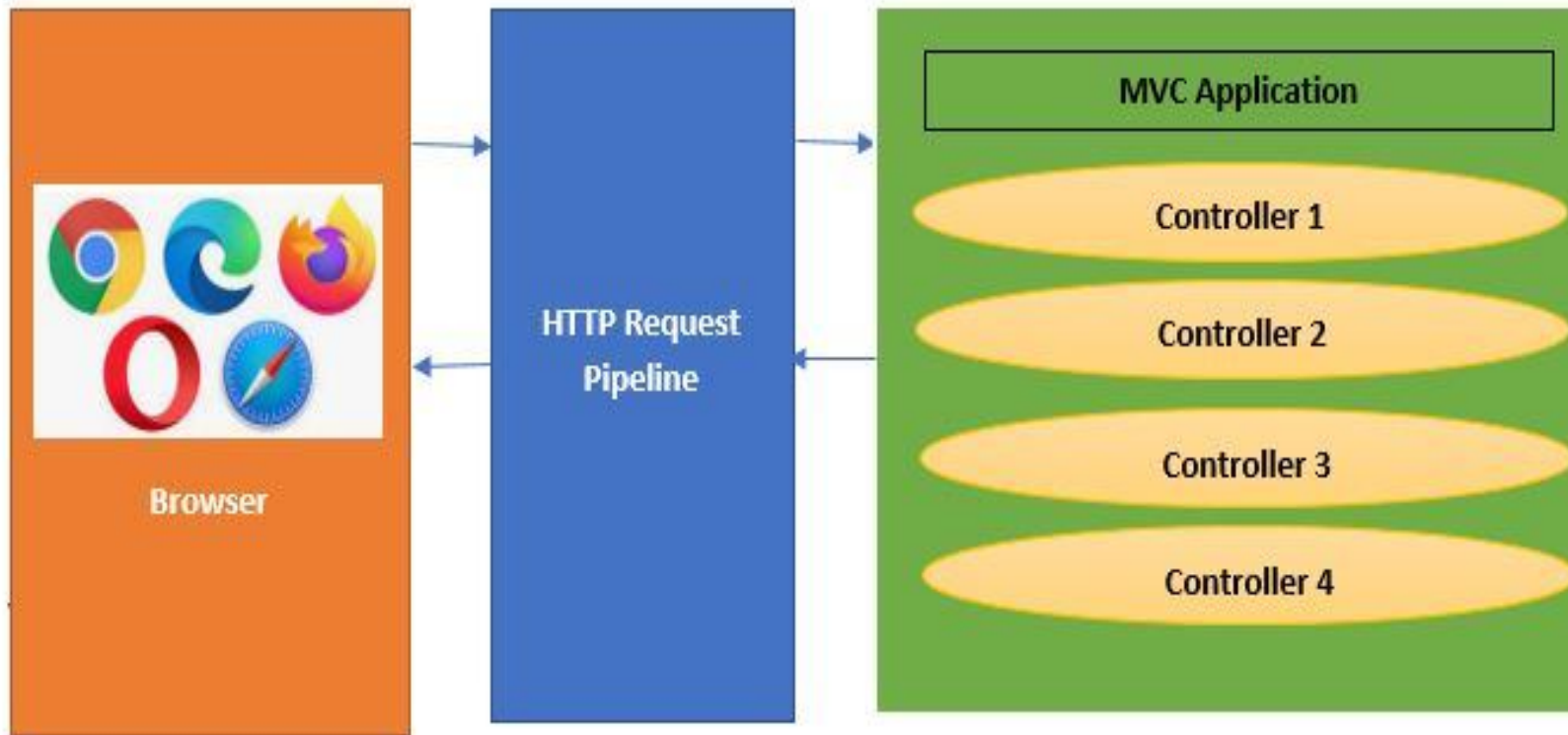


Configure() Method

- You need to include only those middleware components which are required by your application and thus increase the performance of your application.



Request Pipe Line



Request Pipeline

As per the above diagram -

- The browser sent a request to the HTTP request pipeline.
- HTTP pipeline will execute.
- Once HTTP pipeline execution will be completed, the request goes to the MVC application.
- MVC Application executes the request and sends the response back to the HTTP Request pipeline -> Browser.



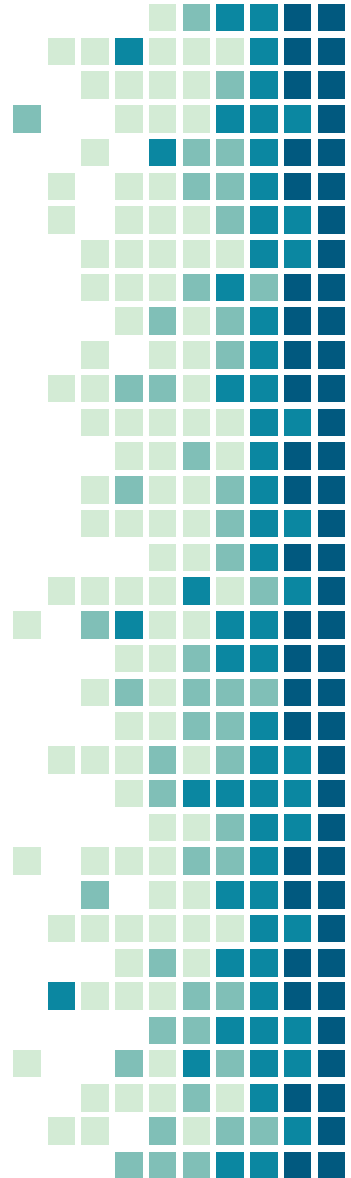
What is Middleware?

- Middleware is new Concept in ASP.NET Core.
- ASP.NET Core Middleware is a fundamental concept in ASP.NET Core applications.
- Middleware can be a function/class/functional unit will perform some specific task.
- Middleware components are the building blocks of the request processing pipeline in an ASP.NET Core application.



What is Middleware?

- They handle specific tasks during request processing, such as authentication, routing, logging, and more.
- Middleware components allow you to add functionalities in a modular and customizable manner.
- Middleware acts as a bridge between the web server and your application code.



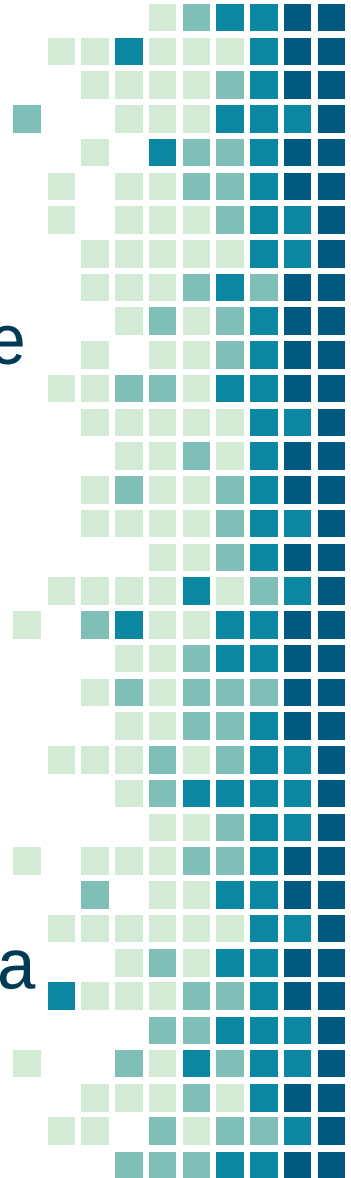
What is Middleware?

- It processes HTTP requests and responses.
- Middleware components are arranged in a pipeline, where each component performs specific tasks.
- The pipeline starts with the web server and ends with your application code.

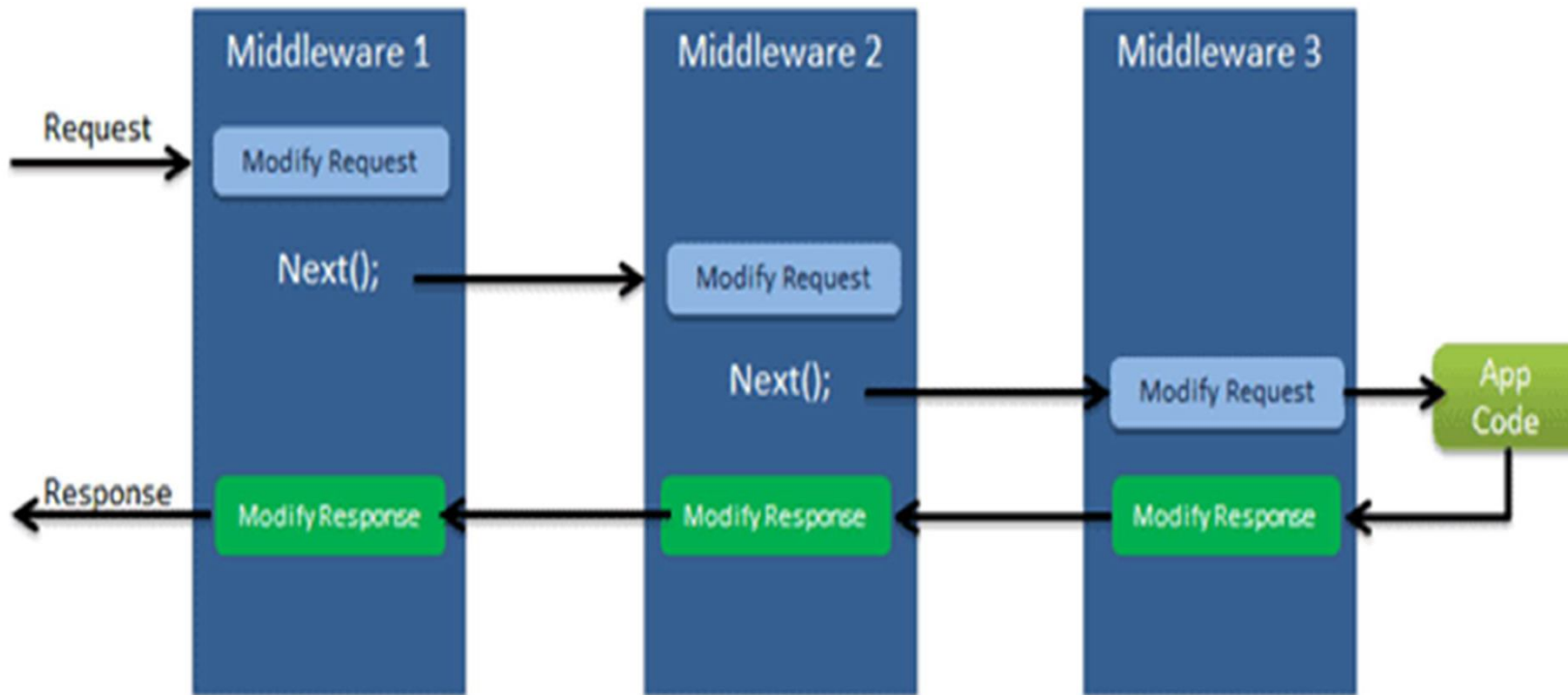


How middleware's work?

- Has Access to both Request & Response
- May simply pass the Request to next Middleware
- May process and then pass the Request to next Middleware
- May handle the request and short circuit the pipeline
- May Process the outgoing Response.
- Middleware's are executed in the order they are added.



What is Middleware?



What is Middleware?

- Configure middleware in the Configure method of the Startup class using IApplicationBuilder instance.
- Middleware's can be built in or can be user defined.
- IApplicationBuilder provides run and use method to create user defined middleware.



What is Middleware?

- Configure middleware in the Configure method of the Startup class using IApplicationBuilder instance.
- Middleware's can be built in or can be user defined.
- IApplicationBuilder provides run and use method to create user defined middleware.



Run Method

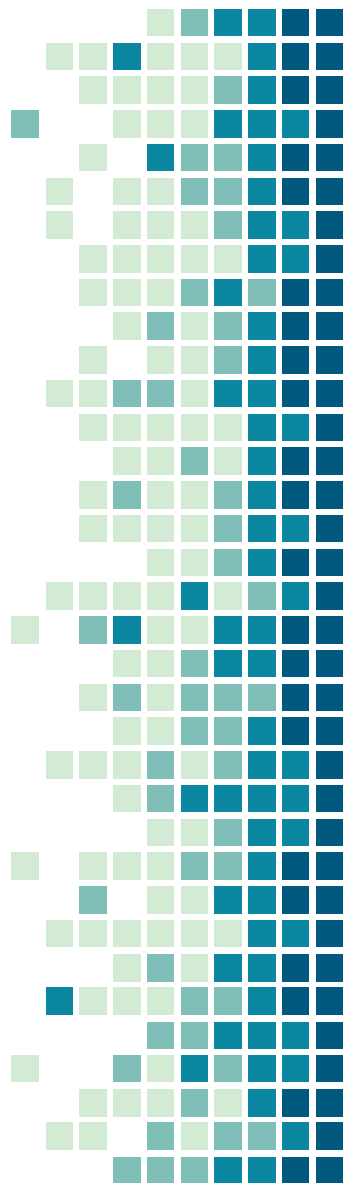
- The Run method is an extension method on IApplicationBuilder & accepts a RequestDelegate as parameter.
- The RequestDelegate is a delegate method which handles the request.
- RequestDelegate stores a methods who has HttpContext object as parameter.
- Run method has limitation that it will short circuit the Request Pipe Line.



Run Method

```
app.Run(async(context)=>{  
    await context.Response.WriteAsync("Hello World!");  
});
```

```
Public void Configure(IApplicationBuilder app,  
IHostingEnvironment env) {  
    app.Run(MyMiddleware);  
}  
  
private Task MyMiddleware(HttpContext context){  
    return context.Response.WriteAsync("Hello World! ");  
}
```



Use Method

- Use method is used to configure multiple middleware.
- Use() method has RequestDelegate as parameter who has two parameters.
- The first parameter is the HttpContext & second parameter is the Func type(generic delegate) that represents next middleware in the pipeline.



Use Method

```
app.Use(async (context, next) => {  
  await context.Response.WriteAsync("Hello World From  
  1st Middleware!");  
  await next(); });
```

THANKS!

Any questions?

You can find me at:

www.ritechpune.com

contact@ritechpune.com

9730010404

